# OAI-PMH for CollectionSpace

## Technical Specification

*Enabling discovery of a CollectionSpace object in an OAI-PMH enabled discovery repository*

Document Status: FINAL
Version: 0.6
Date: August 2026
Source Story: C1
Project: LYRASIS Interoperability Project
Systems: CollectionSpace / OAI-PMH 2.0 / Enabled repositories or discovery layers

# Table of Contents

Table of Contents
Purpose and Scope
Background
How “publishing” works
    Recommendation: reuse, don’t extend
Actors and Deployment Context
System Overview
    Integration Architecture
    Typical URL shapes
    CollectionSpace repositories touched
    Existing components to reuse
    Proposed new components
    Configuration model
    Staff UI touchpoints (cspace-ui.js)
    Data flow: harvest index (recommended)
    OAI protocol surface
    Delete and unpublish semantics
    Dublin Core mapping
    Reference: ArchivesSpace OAI (behavioral, not code reuse)
Configuration Requirements

Behavior Scenarios
Error Scenarios
Other Requirements
Implementation Gaps
Suggested epics

# Purpose and Scope

This specification defines requirements for adding OAI-PMH (Open Archives Initiative Protocol for Metadata Harvesting) provider functionality to CollectionSpace. Adding OAI-PMH enables external discovery systems to harvest CollectionSpace object records and their associated digital content on a configurable schedule.

OAI-PMH 2.0 is the target protocol version. The feature has two components: (1) an OAI-PMH compliant API endpoint exposed by CollectionSpace, and (2) a graphical configuration interface within CollectionSpace allowing administrators and staff users to control which records are harvested and how.

Out of scope:

- Enforcing re-harvesting when data is deleted or added
- Non-OAI-PMH sharing protocols
- Support for OAI-PMH retrieval of procedure or authority records
- Any CollectionSpace records not explicitly flagged for public sharing
- Exporting metadata formats beyond oai_dc

# Background

OAI-PMH is a standard and best practice for sharing metadata records, particularly in libraries and archives. Many discovery systems are OAI-PMH compliant. Many institutions that use CollectionSpace also operate an OAI-PMH compliant discovery layer (such as a digital library, institutional repository, library catalog, or union catalog).

Without native OAI-PMH support in CollectionSpace, institutions must either build custom export pipelines (e.g., exporting to Solr and exposing via the Solr API) or forgo discovery layer integration entirely.

Adding OAI-PMH provider support to CollectionSpace would:

- Remove the need for institution-specific middleware to push CollectionSpace data to discovery systems.

- Make CollectionSpace collections more discoverable alongside library and archival content in shared discovery interfaces.

# How “publishing” works

This is the central question raised in program-team feedback: s*hould OAI be a new "Publish To" target (a Publish to OAI option alongside Publish to browser), or simply a reuse of the existing public/not-public state?* Checked against the current codebase, the recommendation is **reuse the existing public state - do not add an OAI-PMH publish target.** This section documents the evidence so the decision is grounded in how the code actually behaves.

The mechanism is `Publish To (publishTo)`, a **repeating term-list field** backed by the `publishTo` vocabulary. It is present on:

- Collection objects - `collectionobjects_common:publishToList/publishTo`
- Media - `media_common:publishToList/publishTo`

A record is treated as publicly published when its `publishTo` list contains a term whose refName shortId is:

- `all` - label "*All*"
- `cspacepub` - label "*CollectionSpace Public Browser*"

So the public browser's record set is precisely collection objects whose `publishToList.shortid` is in `{all, cspacepub}`. This is the same set the program team already said OAI should mirror.

## Recommendation: reuse, don’t extend

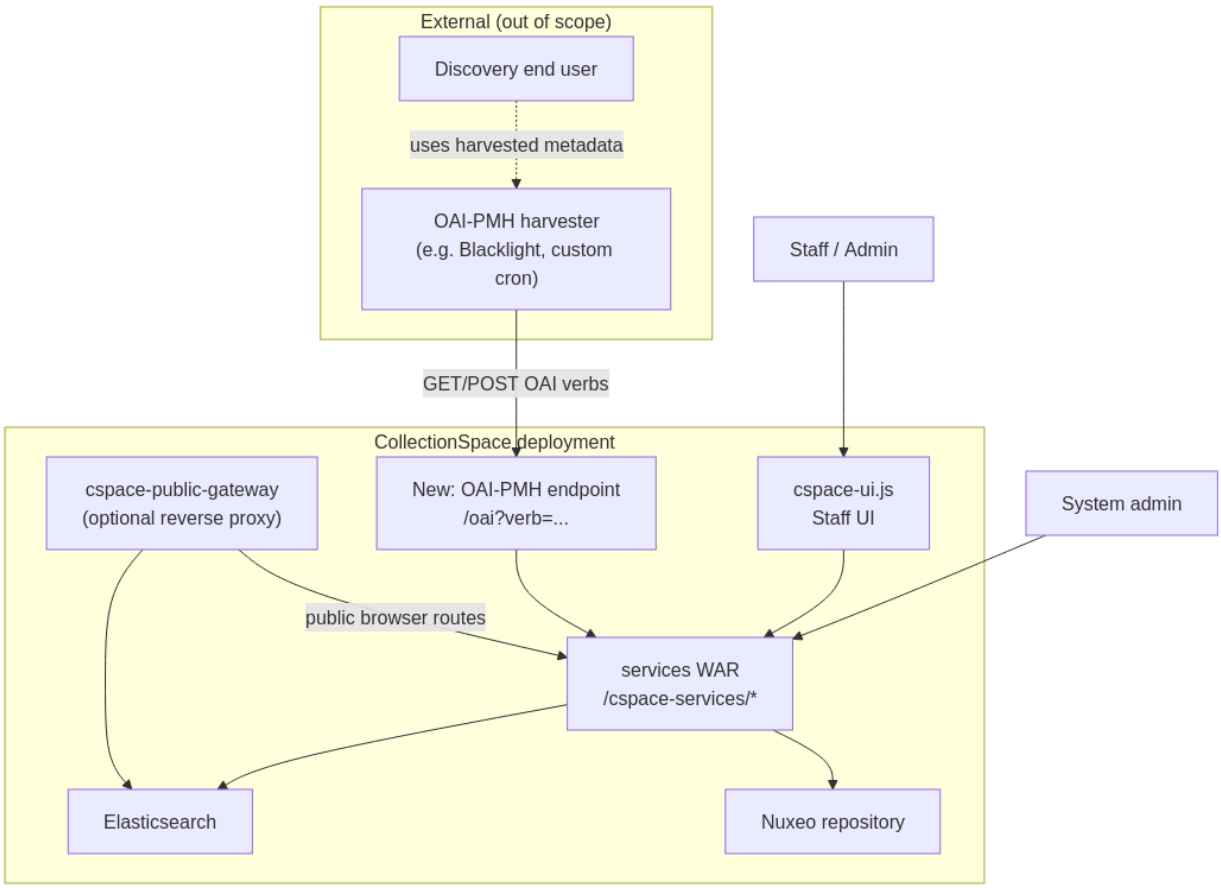
OAI harvest eligibility = the existing public-published state. Concretely:

- No new `publishto` vocabulary term.
- A record is harvestable when it is public (Publish To includes *All* or *CollectionSpace Public Browser*) - the same switch staff already use for the public browser.
- Which shortIds count is **configurable** (mirroring the gateway's `allowedPublishToValues`); default `all,cspacepub`. See `oai.harvestPublishToValues` below.

**Intended consequence**: OAI exposure becomes *coupled* to public-browser exposure - a record cannot be harvested via OAI without also being public in the browser, and vice-versa. The feedback explicitly asks for this single public/not-public switch, so the coupling is a feature, not an accident. If later work needs OAI-only or browser-only publishing, that is when a per-channel

"Publish To" target would be introduced. The standing question in the Gap G-18 - "*refactor Publish To... to a simple publish toggle?*" - is compatible with this: a single toggle would set/clear the public term.

# Actors and Deployment Context




Roles and permissions in CollectionSpace are defined at the local level. The user role that each local repository chooses should have the following permissions.

| Actor Role | Actor type | Responsibility | Other Requirements |
|---|---|---|---|
| CollectionSpace System Administrator | User | Enables or disables access to the feature at the instance level and sets the default configuration | Configurations available only to this role do not need to be exposed in the Staff UI. |

| | | | |
|---|---|---|---|
| CollectionSpace OAI-PMH Administrator (new) | User | Can disable the feature and customize some aspects of it. | Necessitates that a new permission be added to CollectionSpace to designate this level of access.<br><br>Any configuration available to this role must be editable from the Staff UI. |
| CollectionSpace Staff User | User | Creates and edits collection objects. May be able to make objects public thereby exposing them through the OAI-PMH interface. | Staff user should be able to understand what enables OAI-PMH either from documentation or cues in the CSpace GUI |
| Harvester | System | Calls OAI verbs on a schedule; handles resumption tokens and incremental `from`/`until`. | |
| OAI provider module (new) | System | Serves XML responses from harvest index + config; enforces enabled/disabled state. | |
| End User (public) | User | Access successfully harvested collections | |

# System Overview

## Integration Architecture

| Component | Role | Integration Information |
|---|---|---|
| CollectionSpace Application | Source of truth for object metadata and digital object references. | Internal CSpace data model |
| CollectionSpace Gateway | | |
| CollectionSpace OAI-PMH Provider | New API endpoint for CollectionSpace that adheres to the OAI-PMH | OAI-PMH specification; must be accessible at distinct path (e.g. /api/oai) |

| Component | Role | Integration Information |
|---|---|---|
| | 2.0 spec | |
| Collection Space Staff UI | Collection Space User Interface | |
| OAI-PMH Configuration UI | Admin GUI within CollectionSpace for managing endpoint settings. | CollectionSpace admin interface |
| 'Publish To' Field | Staff-facing record-level toggle that marks an object as publishable. | Existing CSpace field which must be refactored to also publish to OAI-PMH endpoint |
| Discovery System / Harvester | External system that periodically sends OAI-PMH requests to the CollectionSpace endpoint and ingests responses. | Standard OAI-PMH 2.0 client (not in scope to build) |

## Typical URL shapes

Assume tenant `museum` and host `https://cs.example.edu`:

| Surface | Example URL | Notes |
|---|---|---|
| Services REST (authenticated) | `https://cs.example.edu/museum/cspace-services/collectionobjects/...` | Existing pattern |
| Public item (anonymous) | `https://cs.example.edu/museum/cspace-services/publicitems/{csid}/{tenantId}/content` | `PublicItemResource` precedent |
| **Proposed OAI base** | `https://cs.example.edu/museum/cspace-services/oai` | Registered in services Web application ARchive (WAR), a packaged Java web application; anonymous when enabled |

| Surface | Example URL | Notes |
|---|---|---|
| Gateway (if used) | `https://cs.example.edu/gateway/museum/cspace-services/oai` | Optional; add route in `cspace-public-gateway` |

## CollectionSpace repositories touched

| Repository | Role in OAI feature | Expected change level |
|---|---|---|
| collectionspace/services | **Primary** - JAX-RS endpoint, mappers, batch jobs, listeners, tenant bindings | **Major** - new `oai` module |
| collectionspace/application | Tenant templates, default properties, Tomcat packaging | **Minor** - defaults in `*-tenant.xml`, profile deltas |
| collectionspace/cspace-ui.js | Staff UI - settings page, Publish, bulk job invoke | **Moderate** - new admin plugin + vocabulary display |
| collectionspace/cspace-public-gateway | Anonymous proxy to services/Elasticsearch (ES) | **Optional minor** - route `/oai` if harvesters must use gateway hostname |
| Nuxeo platform extensions (under `services/`) | Document types, listeners | **Possible** - harvest record doctype or reuse ES-only index |

## Existing components to reuse

CollectionSpace has **no OAI-PMH code today**. These patterns are the closest analogues:

### HTTP / JAX-RS registration

| Component | Location | Reuse for OAI |
|---|---|---|
| `CollectionSpaceJaxRsApplication` | `services/JaxRsServiceProvider/.../Collection` | Register new `OaiResource` singleton |

| Component | Location | Reuse for OAI |
|---|---|---|
| | `SpaceJaxRsApplication.java` | |
| `SecurityInterceptor` | `services/common/.../security/SecurityInterceptor.java` | Whitelist anonymous access on `/oai` when enabled (same pattern as `PublicItemResource.allowAnonymousAccess()`) |
| `*Resource` + `*Client` pair | Each service module | Follow `ExportClient.SERVICE_PATH` convention |

## Public / published content

| Component | Location | Reuse for OAI |
|---|---|---|
| `PublicItemResource` | `services/common/.../publicitem/PublicItemResource.java` | Precedent for **anonymous** read endpoint backed by Nuxeo |
| `PublicItemDocumentModelHandler` | `services/publicitem/../nuxeo/` | URL generation for public media links in Dublin Core `relation`/`identifier` |
| Publish To (`publishto` vocab) | `publishTo` / `publishToList` on collection object, media (cspace-ui.js recordTypes/*/fields.js); public predicate `publishToList.shortid ∈ {all, cspacepub}` (`cspace-public-gateway`, `DefaultESDocumentWriter`) | **Reuse the existing public state** as the harvest gate - no new term |

## Query, export, and bulk update

| Component | Location | Reuse for OAI |
|---|---|---|
| `ExportResource` | `services/export/.../ExportResource.java` | NXQL/query iteration over Nuxeo documents |
| `AbstractDocumentsByQueryIterator` | `services/export/.../` | Walk matching collection objects for harvest index build |
| `BatchResource` + `AbstractBatchJob` | `services/batch/.../` | Bulk enable Publish; scheduled harvest index refresh |
| `ReindexFullTextBatchJob` (example) | `services/batch/.../nuxeo/` | Template for `OaiHarvestIndexBatchJob` |

## Search index

| Component | Location | Reuse for OAI |
|---|---|---|
| `IndexResource` | `services/index/.../IndexResource.java` | `POST /index/{indexid}` reindex hook |
| `IndexDocumentModelHandler` | `services/index/.../` | Per-tenant NXQL for what gets indexed |
| Tenant index bindings | `services/common/.../tenants/tenant-bindings-proto-unified.xml` | Declare new index id (e.g. `oai-harvest`) |
| `Reindex` listener | `services/.../listener/Reindex.java` (source) | **Pattern** for a new listener that, on object change, bumps `collectionspace_core:updatedAt` on related parent objects so they reappear in incremental harvests (see Related-object changes) |

### Authorization

| Component | Location | Reuse for OAI |
|---|---|---|
| `CSpaceAction`, `AuthZ`, `RoleResource`, `PermissionResource` | `services/authorization/*` | New permission e.g. `OAI-PMH Admin` with `ADMIN` on OAI config resource |
| Service authz suffix pattern | `{Module}Client.SERVICE_AUTHZ_SUFFIX` | e.g. `/*/oai/` for config mutations |

### Configuration

| Component | Location | Reuse for OAI |
|---|---|---|
| `tenant-bindings-proto-unified.xml` + profile deltas | `services/common/.../tenants/` | Service binding for OAI module, batch jobs, listeners |
| `{profile}-tenant.xml` | `application/tomcat-main/src/main/resources/` | Instance-level defaults |
| `TenantBindingConfigReaderImpl` | `services/config/.../` | Runtime read of OAI settings |

## Proposed new components

### New Maven module: `services/oai/`

Mirror structure of `export` and `publicitem`:

```
None
services/oai/
  client/
    org/collectionspace/services/client/OaiClient.java
  jaxb/
    oai_config.xsd                    # admin config payload (optional REST CRUD)
    oai_harvest_record.xsd            # internal harvest document shape
  service/
```

```
org/collectionspace/services/oai/
  OaiResource.java                # @Path("/oai") - verb dispatcher
  OaiRequestParser.java           # GET/POST param normalization
  OaiResponseWriter.java          # XML serialization, gzip
  OaiConfigService.java           # read merged tenant + admin config
  OaiHarvestRecordRepository.java # query harvest store (ES or Nuxeo)
  verb/
    IdentifyHandler.java
    ListMetadataFormatsHandler.java
    ListSetsHandler.java          # returns noSetHierarchy
    ListIdentifiersHandler.java
    ListRecordsHandler.java
    GetRecordHandler.java
  OaiResumptionTokenService.java  # token issue/validate/TTL
  mapper/
    OaiMetadataMapper.java        # interface
    OaiDcMapper.java              # oai_dc for collectionobject (per profile)
    ProfileMapperRegistry.java    # anthro, fcart, registration, etc.
  nuxeo/
    OaiHarvestDocumentModelHandler.java   # if harvest docs live in Nuxeo
  batch/
    OaiHarvestIndexBatchJob.java          # rebuild harvest index
    OaiPublishBatchJob.java               # bulk set Publish from query
resources/
  OaiHarvestIndexBatchJob.xml
  OaiPublishBatchJob.xml
```

## Class responsibilities (sketch)

**`OaiResource`** - single entry point; OAI uses query params, not RESTful subpaths:

```java
@Path(OaiClient.SERVICE_PATH)  // "/oai"
public class OaiResource extends AbstractResource {
    public boolean allowAnonymousAccess() { return
configService.isEndpointEnabled(); }

    @GET @POST
    @Produces("application/xml")
    public Response handleOaiRequest(@Context UriInfo uri, MultivaluedMap
formParams) {
```

```
        if (!configService.isEndpointEnabled()) return disabledResponse();
        OaiVerb verb = parser.parseVerb(uri, formParams);
        return verbRouter.dispatch(verb, parser.buildContext(uri, formParams));
    }
}
```

**`isEndpointEnabled`** checks both the Admin's Enabled flag as well as the System Admin's Feature Available flag. If either are false, the feature is disabled.

**`OaiHarvestIndexBatchJob`** - ETL from Nuxeo → harvest store:

```
Java
// Pseudocode: select published collection objects, map to harvest records
// Eligibility reuses the existing public state (the same set as the public
browser):
// the record's Publish To list contains a public term whose refName shortId is
// "all" or "cspacepub". Configurable via oai.harvestPublishToValues (default:
all,cspacepub).
for (DocumentModel doc : findPublicCollectionObjects(from, until)) {
    HarvestRecord rec = mapper.toHarvestRecord(doc);
    rec.setOaiIdentifier(pidFactory.for(doc));
    rec.setDatestamp(doc.getModifiedDate()); // UTC seconds
    harvestRepository.upsert(rec);
}
```

**`ListRecordsHandler`** - read from harvest store, not live Nuxeo:

```
Java
Page<HarvestRecord> page = repository.findPublished(
    metadataPrefix, from, until, set, resumptionToken, maxRecords);
if (page.isEmpty()) return error("noRecordsMatch");
OaiXml records = page.items().stream()
    .map(r -> xml.record(r.identifier(), r.datestamp(), mapper.toMetadata(r)))
    .collect(OaiXml.listRecords());
if (page.hasMore()) records.setResumptionToken(tokenService.issue(page));
return Response.ok(records).build();
```

### Files to modify (existing)

| File | Change |
|---|---|
| `services/pom.xml` | Add `<module>oai</module>` |
| `CollectionSpaceJaxRsApplication.java` | `singletons.add(new OaiResource());` or `addResourceToMapAndSingletons` |
| `tenant-bindings-proto-unified.xml` | OAI service binding, batch jobs, optional listener, index definition |
| Profile `*-tenant-bindings.delta.xml` | Register profile-specific `OaiDcMapper` |

## Configuration model

### Tier 1 - System administrator (instance / tenant template)

Stored in **tenant bindings XML** and/or `{profile}-tenant.xml` in the **application** repo. Not editable in Staff UI.

| Key | Example | Purpose |
|---|---|---|
| `oai.featureAvailable` | `true` | Master switch - without this, OAI code paths are inactive |
| `oai.baseUrl` | `https://cs.example.edu/museum/cspace-services/oai` | Canonical `baseURL` in Identify (may be computed from request if unset) |
| `oai.protocolVersion` | `2.0` | Identify response |
| `oai.deletedRecord` | `no` | Signal to harvester that CollectionSpace does not maintain records of “deleted” (unpublished) records. |
| `oai.granularity` | `YYYY-MM-DDThh:mm:ssZ` | Identify response |

| Key | Example | Purpose |
|---|---|---|
| `oai.earliestDatestamp` | `YYYY-MM-DDThh:mm:ssZ` | Identify - computed from harvest index min datestamp unless overridden |
| `oai.maxRecordsPerRequest` | `100` | Resumption token threshold |
| `oai.resumptionTokenTtlSeconds` | `3600` | Token cache TTL |
| `oai.compressionSupported` | `true` | gzip response encoding |
| `oai.harvestIndexId` | `oai-harvest` | ES index name or Nuxeo doc type namespace |
| `oai.harvestPublishToValues` | `all,cspacepub` | Publish To shortIds that make a record harvestable; mirrors the gateway's `allowedPublishToValues`. Default = the public-browser set (i.e. OAI eligibility == public state) |
| `oai.harvestRefreshCron` | *(external)* | Document that ops scheduler invokes batch job - CS has no built-in cron |
| `oai.metadataFormats` | `oai_dc` | ListMetadataFormats source of truth |
| `oai.setSupportEnabled` | `false` | Disable OAI Set Support |

**Storage mechanism:** extend tenant service binding for a new `OaiClient.SERVICE_NAME` in `tenant-bindings-proto-unified.xml`, properties read at runtime via existing `ServiceMain.getTenantBindingConfigReader()`.

## Tier 2 - OAI-PMH Administrator (Staff UI)

Stored in **Nuxeo configuration document** or **dedicated config record** exposed via authenticated REST (pattern used by other admin settings). Editable in Staff UI when user holds new permission.

| Key | Purpose |
|---|---|
| `oai.enabled` | Administrator toggle - endpoint returns 503/disabled when false |
| `oai.repositoryName` | Identify |
| `oai.repositoryDescription` | Identify `<description>` |
| `oai.adminEmail` | Identify (required) |
| `oai.includeMediaLinks` | Whether DC includes thumbnail/public URLs (`Supported Media` in Configuration Requirements) |
| `oai.publicBaseUrl` | Base for `relation` link to public browser record |

**REST shape (illustrative):**

GET /cspace-services/oai/config → 200 + OAI config JSON/XML (auth required)

PUT /cspace-services/oai/config → update admin-tier settings

POST /cspace-services/oai/config/test → optional Identify dry-run for admins

## Record-level eligibility

| Field | Where | Rule |
|---|---|---|
| `publishTo` / `publishToList` | Collection object (and media, for media links) | Must include a **public** term (refName shortId `all` or `cspacepub`) - the same state that publishes to the public browser. No OAI-specific term |

| Field | Where | Rule |
|---|---|---|
| `collectionspace_core:updatedAt` / record audit | Nuxeo | Becomes OAI **datestamp** (UTC, seconds).<br><br>Must also be bumped on related records when a related object's publish/eligibility state changes (see below), so incremental `from`/`until` harvests pick them up |

Bulk update: marking records public is the **existing publish-to-public workflow.** A bulk data update can set the public Publish To term. Because eligibility reuses the public state, this does not need to be an OAI-specific job.

### Related-object changes and incremental reharvest

Harvesters discover changed records via the OAI **datestamp**, which is derived from `collectionspace_core:updatedAt`. That field is updated when the **object record itself** is edited. It is **not** updated when only a *related* object changes, for example, when a related media record's public Publish To term is cleared (an image taken down). In that case the parent collection object still carries a `dc:relation` (or similar) to content that is no longer public, but its own `updatedAt` is unchanged, so a subsequent incremental harvest will **not** re-fetch it.

This is a known limitation of the public browser today: un-publishing a related object does not refresh the parent. For OAI, the requirement is that the harvester must learn the parent needs reharvesting on the next run so the stale relation is no longer served.

**Recommended approach (high level):** copy and adapt the existing Nuxeo Reindex listener pattern. That listener, when an object changes, tells Elasticsearch to reindex related documents. A parallel listener should, when an object changes (including publish/unpublish), update `collectionspace_core:updatedAt` on all objects that **relate to it**. Those parents then carry a newer datestamp, appear in the next `from`/`until` window, and are remapped in the harvest index without the harvester needing special related-object logic.

Exact relation traversal and listener wiring are left to implementation; the requirement is that **related-object publish-state changes propagate into parent datestamps**.

## Staff UI touchpoints (`cspace-ui.js`)

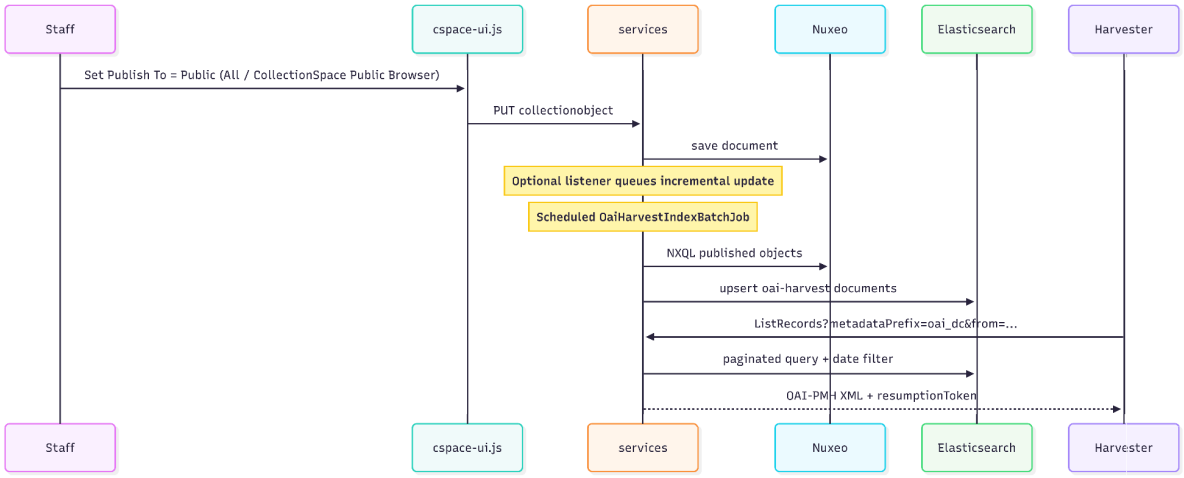
| Location | Change |
|---|---|
| **Administration → OAI-PMH Settings** (new) | New invocable plugin under admin/tools; mirrors ArchivesSpace "Manage OAI-PMH Settings" |
| **Publish To** field | **No change** - a record becomes harvestable when marked public (Publish To = *All* or *CollectionSpace Public Browser*), the same control already used for the public browser |
| **Bulk data update** | Reuse the existing publish-to-public bulk workflow (set the public Publish To term from a search query); no OAI-specific job required |
| **Record editor cue** | Optional help text clarifying that public records are also exposed via OAI-PMH (BS03: Staff user marks multiple records as eligible for OAI-PMH harvesting) |

Proposed plugin path: `src/plugins/invocables/oaiSettings/` (or under existing admin plugin structure).

Permission gate: hide settings unless the user has new **OAI-PMH Administrator** permission.

## Data flow: harvest index (recommended)

Meeting notes and performance requirements point to **decoupling harvest reads from live Nuxeo queries**:




## Harvest record document (logical)

Internal shape stored in ES (or Nuxeo sidecar documents):

```json
{
  "csid": "abc-123",
  "oaiIdentifier": "oai:cs.example.edu:museum/abc-123",
  "datestamp": "2026-06-01T14:30:00Z",
  "metadataPrefix": "oai_dc",
  "profile": "anthropology",
  "metadataXml": "<oai_dc:dc>...</oai_dc:dc>"
}
```

Pre-computing `metadataXml` at index time avoids mapping cost on every harvest request; remapping requires a reindex job after code/config changes.

**Alternative (simpler, higher load):** query Nuxeo directly on each ListRecords - acceptable only for small repositories; not recommended as default.

# OAI protocol surface

| Verb | Behavior | Handler |
|---|---|---|
| `Identify` | Repository metadata from config | `IdentifyHandler` |

| Verb | Behavior | Handler |
|---|---|---|
| `ListMetadataFormats` | At minimum `oai_dc` | `ListMetadataFormatsHandler` |
| `ListSets` | Error `noSetHierarchy` | `ListSetsHandler` |
| `ListIdentifiers` | Headers from harvest index | `ListIdentifiersHandler` |
| `ListRecords` | Headers + metadata | `ListRecordsHandler` |
| `GetRecord` | Single record by OAI identifier | `GetRecordHandler` |

| Requirement | Implementation note |
|---|---|
| GET and POST | Both hit `OaiResource.handleOaiRequest` |
| Persistent identifiers | Build from tenant + CSID: `oai:{host}:{tenant}/{csid}` |
| Deleted records | `deletedRecord=no`; CollectionSpace will not send records when *Publish To* cleared; a diff to identify and delete stale records on the harvester side is up to the harvester |
| Resumption tokens | `OaiResumptionTokenService` - in-memory or Nuxeo/Redis cache; `badResumptionToken` if expired |
| Compression | `OaiResponseWriter` wraps XML with gzip when `Accept-Encoding` allows |

## Delete and unpublish semantics

Record availability is determined by the `publishTo` field. Published records are harvestable. Unpublished records are not. CollectionSpace will not maintain data about “deleted” or unpublished records for the harvester.

When a record is deleted or unpublished from CollectionSpace, it will no longer be harvestable. It is up to the harvester to compare records against its index, identify stale records, and remove them from the harvesting repository. While harvesters may rely on deleted record information from repositories that offer this data, a good practice for any harvester would be to implement a sync process using ListIdentifiers data if they allow harvesting from repositories that signal `deletedRecord=no`.

# Dublin Core mapping

| Source | Direction |
|---|---|
| Internal DC mapping draft (Confluence) | **Authoritative** for field choices |

## Fields Common to All Profiles

No dc fields are required. All records must have object number which is the only required field in CSpace and should be the only required field for publishing to OAI.

| CollectionSpace Element | DC Element | Notes |
|---|---|---|
| Title | title | |
| objectProductionPerson<br>objectProductionOrganization<br>objectProductionPeople | creator | Default value: objectProductionPerson<br>If Person is blank, then Org<br>If Org is blank, then People |
| objectProductionDate | date | Pull earliest/latest scalar values from the date details and concatenate them with '/'. |
| objectProductionPlace | coverage | |
| contentConcept | subject | |
| briefDescription | description | |
| descriptionLevel:Item<br>descriptionLevel:Group of Items | type | Value: Physical Object |
| dimensionSummary | format | |

| | | |
|---|---|---|
| objectNumber | identifier | |
| rightStatement | rights | |
| Repository Name (OAI-PMH Configuration field) | publisher | Repository name |
| titleLanguage | language | |
| objectHistoryNote | provenance | |

### Contributor Field (not required)

| CSpace Profile | CollectionSpace Element | DC Element | Notes |
|---|---|---|---|
| Anthropology | fieldCollector | contributor | |
| Fine Arts | none | none | |
| Local History | none | none | |
| Public Art | Gap G-16 | Gap G-16 | |

Dublin Core mappings will **not be user-configurable**; no arbitrary XSLT upload.

Implementation: one `OaiDcMapper` subclass or profile section per CollectionSpace profile; registered in `ProfileMapperRegistry` at startup from tenant profile id.

Media links: include public thumbnail URL in `dc:relation` when `oai.includeMediaLinks` is true.

All repeated fields or values in the map will be passed to the harvester. How the harvester displays repeated values is out of scope for the feature (and fully up to the harvester).

## Reference: ArchivesSpace OAI (behavioral, not code reuse)

| AS component | CS analogue |
|---|---|
| `ArchivesSpaceOAIRepository` | `OaiResource` + verb handlers |
| `ArchivesSpaceOAIRecord` | `HarvestRecord` |

| AS component | CS analogue |
|---|---|
| `ArchivesSpaceResumptionToken` | `OaiResumptionTokenService` |
| `oai/mappers/oai_dc.rb` | `OaiDcMapper.java` |
| System → Manage OAI-PMH Settings | Administration → OAI-PMH Settings |
| Separate OAI port (8082) | Shared services WAR path `/oai` (simpler ops) |

# Configuration Requirements

All of the following must be configurable by the CollectionSpace System Administrator. Some elements may be overridden by a CollectionSpace Administrator as indicated. Anything that is available to the CollectionSpace Administrator role may be configurable through the UI.

System Admin settings are instance/tenant configuration (not Staff UI); storage and runtime access are defined in Configuration model - Tier 1. Staff-UI-overridable settings follow Tier 2.

| Function | Role Access | Notes |
|---|---|---|
| Feature Available | System Admin | Feature flag to enable the feature itself; The availability of feature must be able to be turned on or off at the system admin level. |
| Protocol Version | System Admin | OAI protocol version supported. Only 2.0 for now. Include for future proofing |
| PID Source | System Admin | For future proofing, we probably want to make it possible to define different approaches to PIDs. For the initial version PID URI comes from CSID. |
| ResumptionToken Expiration Seconds | System Admin | Defines the cache TTL |
| Max Records Per Request | System Admin | Limit on number of records that can be returned in one response (more than this will trigger use of a resumption token) |
| DateStamp Granularity | System Admin | Should be second, but configurable just in case |

| Function | Role Access | Notes |
|---|---|---|
| Earliest DateStamp | System Admin | This should be the earliest possible datestamp in the system |
| Compression Support | System Admin | Whether or not the system supports compression (gzipping) of results - should be yes by default. |
| Set Support | System Admin | For future proofing, initially not supported |
| Supported Metadata Formats | System Admin | Will require related mapping configuration for each format |
| Supported Media | System Admin | Should be possible to use this to eliminate media links entirely, or to specify which derivatives to include links to (thumbnails, etc.) |
| Disabled | System Admin, Admin | Ability for Administrator to disable the feature even if it's available at the system level. |
| Repository Name | System Admin, Admin | Human-readable name returned in the Identify response (e.g., 'Example Institution CollectionSpace'). |
| Repository Description | System Admin, Admin | Human-readable description of the repository contents (for inclusion in Identify Response) |
| Admin Email | System Admin, Admin | Returned in the Identify response as required by OAI-PMH spec. |

# Behavior Scenarios

## BS01: Administrator enables and configures OAI-PMH in CollectionSpace

| Step | Description |
|---|---|
| Given | The CollectionSpace instance has been configured with the OAI-PMH feature set to available. |

| Step | Description |
|---|---|
| | The administrator is logged into CollectionSpace with OAI-PMH Administrator access. |
| When | The administrator navigates to the OAI-PMH Settings page. |
| | The administrator enables the OAI-PMH endpoint and edits required configuration fields and field mapping details. |
| | The administrator saves the configuration. |
| Then | The CollectionSpace OAI-PMH endpoint becomes active and responds to OAI-PMH verb requests. |
| | The Identify verb returns the configured repository name, admin email, and policy settings. |

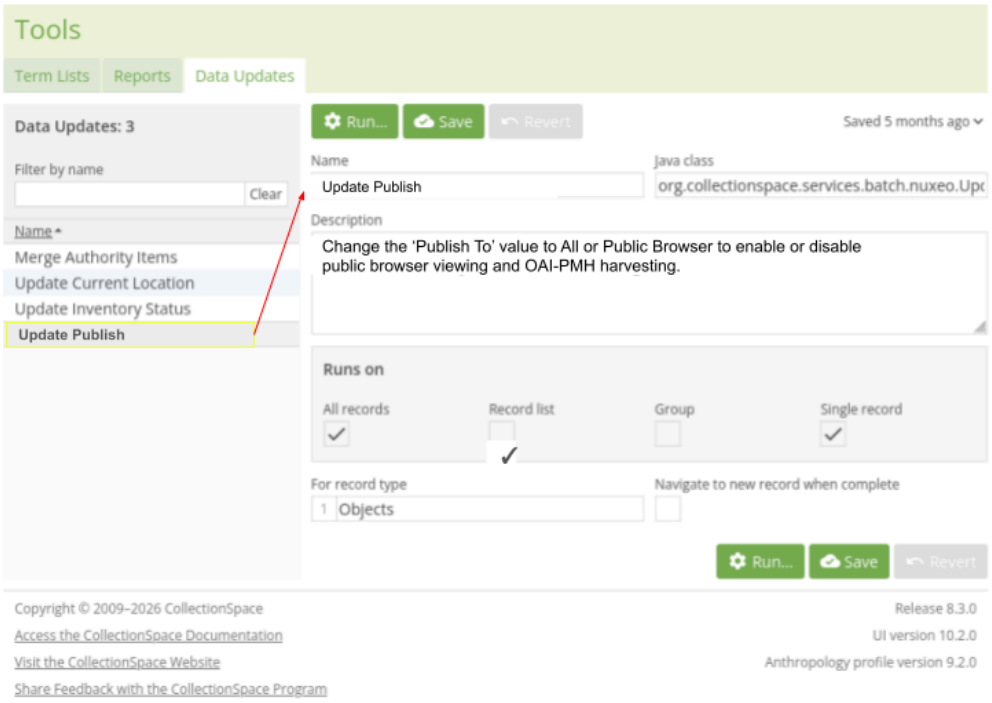
## BS02: Staff user marks a record as eligible for OAI-PMH harvesting

| Step | Description |
|---|---|
| Given | An object record exists in CollectionSpace. |
| | The staff user has write access to the record. |
| When | The staff user opens the record in CollectionSpace. |
| | The staff user Publishes the record to All or CollectionSpace Public Browser |
| | The staff user saves the record. |
| Then | The record is flagged as harvestable. |
| | The record's datestamp is updated to the save timestamp, enabling incremental harvesting via from/until parameters. |

## BS03: Staff user marks multiple records as eligible for OAI-PMH harvesting

| Step | Description |
|---|---|
| Given | Object records exist in CollectionSpace. |
| | The staff user has write access to the records. |

| Step | Description |
|---|---|
| When | The staff user opens the Tools menu and navigates to the Data Updates tab. |
| | The staff user selects Update Publish |
| | The user fills out the options to Publish the records to All or CollectionSpace Public Browser |
| | The user clicks “Run.” |
| Then | The records are flagged as harvestable. |
| | The records’ datestamp is updated to the current time, enabling incremental harvesting via from/until parameters. |




## BS04: Staff user deletes a record that was previously harvested

| Step | Description |
|---|---|
| Given | An object record exists in CollectionSpace. |
| | The staff user has delete access to the record. |
| When | The staff user opens the record in CollectionSpace. |
| | The staff user clicks the Delete button and confirms deletion. |

| Step | Description |
|---|---|
| Then | The record's datestamp is updated to the date and time of deletion. |
| | The record no longer appears in the Public Browser. |
| | Harvesters no longer harvest the deleted record in future harvests, but removing the stale record from the repository is dependent on the harvesting repository's features. |

## BS05: OAI-PMH List Records Request is Fulfilled

| Step | Description |
|---|---|
| Given | The CollectionSpace OAI-PMH endpoint is active. |
| | One or more object records are Published to All or CollectionSpace Public Browser |
| | The discovery system harvester is configured with the CollectionSpace OAI-PMH endpoint URL. |
| When | The discovery system sends a ListRecords (or ListIdentifiers) request with metadataPrefix=oai_dc and the request includes from/until parameters for incremental harvest. |
| Then | CollectionSpace returns an OAI-PMH XML response containing oai_dc records for all published objects (filtered by date if from/until was provided). |
| | Each record includes a unique OAI identifier, datestamp, and mapped Dublin Core metadata, including links to the Media URL and Public Browser URL if present. |
| | If more records exist than fit in one response, a resumptionToken is included and subsequent requests with that token return the next page. |
| | Harvested metadata is displayed in the discovery system within the latency defined by the harvester's schedule (typically ≤ 24 hours after the scheduled harvest). |

## BS06: Discovery system administrator configures the harvester (CollectionSpace requirements)

This scenario describes what CollectionSpace must provide to enable configuration in a generic OAI-PMH harvesting tool. The discovery system UI varies and is not in scope to specify.

| Step | Description |
|---|---|
| Provided by CSpace | A publicly accessible OAI-PMH endpoint URL (e.g., https://cs.example.edu/oai). |
| Provided by CSpace | A valid Identify response including repository name, admin email, and granularity. |
| Provided by CSpace | A valid ListMetadataFormats response listing supported prefixes (minimum: oai_dc). |
| Provided by CSpace | A valid ListSets response if set-based filtering is required. |
| Provided by CSpace | Resumption tokens for ListRecords/ListIdentifiers responses exceeding the configured page size. |
| Provided by CSpace | Authentication for the OAI-PMH endpoint: The CollectionSpace endpoint is publicly accessible without further authentication. |

# Error Scenarios

## ES01: OAI-PMH endpoint is not enabled

| Step | Description |
|---|---|
| Given | The OAI-PMH integration is configured but the OAI-PMH Enabled toggle is off. |
| When | A harvester sends a request to the CollectionSpace OAI-PMH endpoint URL. |
| Then | CollectionSpace returns HTTP 503 or an OAI-PMH error response with code 'badVerb' or a custom message indicating the service is inactive. |

## ES02: Harvest returns no records (empty repository)

| Step | Description |
|---|---|
| Given | The OAI-PMH endpoint is active but no records are currently published. |
| When | A harvester sends a ListRecords request. |
| Then | CollectionSpace returns a valid OAI-PMH response with error code 'noRecordsMatch' as specified by the OAI-PMH 2.0 protocol. |

## ES03: Harvest fails due to server error

| Step | Description |
|---|---|
| Given | The OAI-PMH endpoint is active. |
| When | A server-side error occurs while processing a harvest request (e.g., database timeout, configuration error). |
| Then | CollectionSpace returns an appropriate HTTP 5xx response or OAI-PMH error document. |
| | The error is written to CollectionSpace server logs. |

## ES04: Record missing required Dublin Core fields

All records must have object number which is the only required field in CSpace and should be the only required field for publishing to OAI.

| Step | Description |
|---|---|
| Given | A staff user sets a record's 'Publish To' status to Public Browser. |
| And | The record is missing one or more fields required by the configured Dublin Core mapping (e.g., no title, no date). |
| Then | Include the record with empty DC elements. |

## ES05: Harvester requests a set

| Step | Description |
|---|---|
| Given | The OAI-PMH endpoint is active but no records are currently published. |
| When | A harvester sends a ListRecords request that includes Sets. |
| Then | CollectionSpace returns a valid OAI-PMH response with error code '`noSetHierarchy` ' as specified by the OAI-PMH 2.0 protocol. |

# All Defined Error Scenarios by Verb

## ListIdentifiers

- `badArgument` - The request includes illegal arguments or is missing required arguments.
- `badResumptionToken` - The value of the `resumptionToken` argument is invalid or expired.
- `cannotDisseminateFormat` - The value of the `metadataPrefix` argument is not supported by the repository.
- `noRecordsMatch`- The combination of the values of the `from`, `until`, and `set` arguments results in an empty list.
- `noSetHierarchy` - The repository does not support sets.

## ListMetadata

- **`badArgument`** - The request includes illegal arguments or is missing required arguments.
- **`idDoesNotExist`** - The value of the `identifier` argument is unknown or illegal in this repository.
- **`noMetadataFormats`** - There are no metadata formats available for the specified item.

## ListRecords

- **`badArgument`** - The request includes illegal arguments or is missing required arguments.
- **`badResumptionToken`** - The value of the `resumptionToken` argument is invalid or expired.
- **`cannotDisseminateFormat`** - The value of the `metadataPrefix` argument is not supported by the repository.
- **`noRecordsMatch`** - The combination of the values of the `from`, `until`, `set` and `metadataPrefix` arguments results in an empty list.
- **`noSetHierarchy`** - The repository does not support sets.

## ListSets

- **`badArgument`** - The request includes illegal arguments or is missing required arguments.
- **`badResumptionToken`** - The value of the `resumptionToken` argument is invalid or expired.
- **`noSetHierarchy`** - The repository does not support sets.

### GetRecord

- `badArgument` - The request includes illegal arguments or is missing required arguments.
- `cannotDisseminateFormat` - The value of the `metadataPrefix` argument is not supported by the item identified by the value of the `identifier` argument.
- `idDoesNotExist` - The value of the `identifier` argument is unknown or illegal in this repository.

### Identify

- `badArgument` - The request includes illegal arguments.

# Other Requirements

## Documentation

Documentation for this feature should include:

- How to configure your repository and enable OAI-PMH
- Scope of harvested records
- How deleted records are handled
- Required data
- Troubleshooting
- Governance, support, and contact information
- Accessibility features and compliance
- Reference documentation, such as field definitions

The documentation may include screenshots and examples.

## Governance

The feature design should be reviewed, further edited, and if agreed upon, formally accepted by the CollectionSpace development team.

Maintenance of this feature after the implementation phase will fall to the CollectionSpace development team.

## Error Reporting

Errors are reported in CollectionSpace server logs.

## Performance and Scalability

No performance issues if the harvester is not running. If the harvester is running, it will use ElasticSearch index.

# Implementation Gaps

| # | Gap | Description | Owner |
|---|---|---|---|
| **G-16** | Metadata Map finalizations | Product team will add mapping for public art | Program Manager, CSpace team |
| **G-17** | Consideration to bypass mediaURL gateway | mediaURL is sent to CSpace services. The gateway may be necessary in this loop. If we bypass the gateway for OAI, would the URL still work? | CSpace team, Implementation team |
| **G-18** | Refactor publishto: field | The spec is designed to act on publishto: values “All” or “Public Browser.” The product team discussed changing and refactoring this field. Further discussion and development plans for this are out of scope for the initial project and could be addressed by the program team separately. | CSpace team |

# Suggested epics

These are units of work, **not sequential phases** - the whole feature is a single delivery. The numbering is for reference only; several epics can proceed in parallel once Epic 1 establishes the harvest index (e.g. Admin UI and Staff workflow), and Epic 5 is the natural "later" work if the program team chooses to extend beyond the baseline.

| Epic | Deliverable | Repos |
|---|---|---|
| **0 - Spike** | Identify + ListMetadataFormats on static config; anonymous routing | `services` |
| **1 - Core harvest** | Harvest index job, ListRecords/ListIdentifiers/Ge | `services`, `application` |

| Epic | Deliverable | Repos |
|---|---|---|
| | tRecord, `oai_dc` for one profile | |
| **2 - Admin UI** | Settings page, permission, enable/disable toggle | `cspace-ui.js`, `services` |
| **3 - Staff workflow** | Reuse public Publish To state, bulk publish (existing workflow) | `services`, `cspace-ui.js` |
| **4 - Hardening** | Compression, resumption token edge cases, logging, additional profiles | `services` |
| **5 - Sets & formats** | ListSets, optional metadata prefixes | `services` |